

UNIT I – NODE.JS

Introduction to Server-side programming – multi-tier architecture - Node.js architecture –
npm – Development environment – API

INTRODUCTION TO SERVER-SIDE PROGRAMMING

Server-side programming refers to the process of creating and executing code on a server that interacts with client-side applications and delivers dynamic content over the internet. In this context, the server typically refers to a web server that hosts websites or web applications.

Difference Between Client Side and Server Side Programming

- **Client-Side Programming:** Code that runs in the user's browser, interacting directly with the user interface.
- **Server-Side Programming:** Code that runs on the server, handling data storage, processing, and business logic.

Client-side scripting	Server-side scripting
Source code is visible to the user.	Source code is not visible to the user because its output of server-side is an HTML page.
Its main function is to provide the requested output to the end user.	Its primary function is to manipulate and provide access to the respective database as per the request.
It usually depends on the browser and its version.	In this any server-side technology can be used and it does not depend on the client.
It runs on the user's computer.	It runs on the webserver.
There are many advantages linked with this like faster response times, a more interactive application.	The primary advantage is its ability to highly customize, response requirements, access rights based on user.
It does not provide security for data.	It provides more security for data.
It is a technique used in web development in which scripts run on the client's browser.	It is a technique that uses scripts on the webserver to produce a response that is customized for each client's request.
HTML, CSS, and javascript are used.	PHP, Python, Java, Ruby are used.
No need of interaction with the server.	It is all about interacting with the servers.
It reduces load on processing unit of the server.	It surge the processing load on the server.

Here are some key points to note about server-side programming:

1. **Server-Side Languages:** There are several programming languages commonly used for server-side development, including but not limited to:

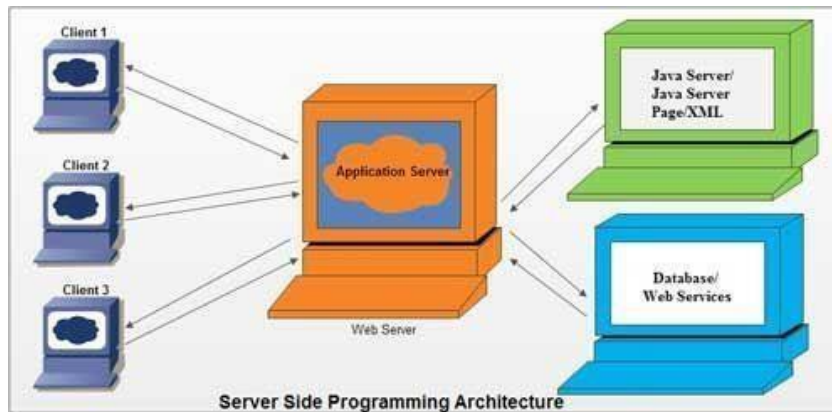
- **PHP:** A popular language for web development due to its simplicity and wide adoption.
- **Python:** Known for its readability and versatility, Python is widely used in web development.
- **Ruby:** Often associated with the Ruby on Rails framework, it emphasizes developer productivity.
- **Java:** A versatile language used for a wide range of applications, including server-side development.

- C#: Primarily used with Microsoft technologies, it's a powerful language for building web applications.
2. Web Frameworks: Web frameworks provide a foundation for server-side programming by offering pre-built libraries, tools, and abstractions that simplify development. Some popular web frameworks include:
- Django (Python): A high-level framework that emphasizes rapid development and follows the model-view-controller (MVC) architectural pattern.
 - Ruby on Rails (Ruby): An opinionated framework that promotes convention over configuration and emphasizes developer productivity.
 - Laravel (PHP): A PHP framework known for its elegant syntax, expressive syntax, and rich ecosystem.
 - Express.js (JavaScript/Node.js): A minimalist framework for building web applications using JavaScript on the server-side.
3. Server-Side Architecture: Server-side programming typically involves designing and implementing the architecture for handling incoming requests and generating appropriate responses. Key components of server-side architecture include:
- Routing: Defining routes to map incoming requests to specific functions or methods in the server-side code.
 - Data Persistence: Interacting with databases or other storage systems to retrieve, update, or store data.
 - Business Logic: Implementing the core logic and algorithms that process requests and generate responses.
 - Security: Implementing authentication, authorization, and other security measures to protect sensitive data and resources.
 - Integration: Connecting with external services or APIs to perform tasks such as payment processing or sending emails.
4. Dynamic Content Generation: Server-side programming enables the creation of dynamic web pages or applications. By generating content on the server before delivering it to the client, developers can create personalized and interactive experiences for users. This can involve dynamically generating HTML, rendering templates, interacting with databases, or consuming external APIs.
5. Client-Server Communication: Server-side programming involves handling client requests and sending appropriate responses. This can include handling HTTP methods like GET, POST, PUT, and DELETE, processing form data, handling file uploads, and sending JSON or XML responses.
6. Scalability and Performance: As server-side applications handle multiple concurrent requests, scalability and performance are crucial considerations. Techniques like caching, load balancing, asynchronous programming, and database optimization are commonly employed to ensure efficient and responsive server-side applications.

Why Server Side Programming

Though it is technically feasible to implement almost any business logic using client-side programs, logically or functionally it serves no purpose when it comes to enterprises application (e.g., banking, air ticketing, e-shopping, etc). To further explain, going by the client-side programming logic; a bank having 10,000 customers would mean that each customer would have a copy of the program(s) in his or her PC (and now even mobiles) which translates to 10,000 programs! In addition, there are issues like security, resource pooling, concurrent access and manipulations to the database which simply cannot be handled by client-side programs. The answer to most of the issues cited above is- “Server-Side Programming”. Figure illustrates the

Server-Side Architecture in the simplest way.



Advantages of Server-Side Programs

Some of the advantages of Server-Side programs are as follows:

- i. All programs reside in one machine called server. Any number of remote machines (called clients) can access the server programs.
- ii. New functionalities to existing programs can be added at the server side which the clients can take advantage of without having to change anything.
- iii. Migrating to newer versions, architectures, design patterns, adding patches, switching to new databases can be done at the server side without having to bother about client's hardware or software capabilities.
- iv. Issues relating to enterprise applications like resource management, concurrency, session management, security and performance are managed by the server side applications.
- v. They are portable and possess the capability to generate dynamic and user-based content (e.g., displaying transaction information of credit card or debit card depending on user's choice).

Types of Server-Side Programs

The following are the different types of server-side programs:

1. Active Server Pages (ASP)
2. Java Servlets
3. Java Server Pages (JSP)
4. Enterprise JavaBeans (EJB)
5. PHP

Node.js, often referred to as just Node, is a powerful tool that can run JavaScript applications on both the server side as well as the client side. Node.js can be used to write static file servers, Web application frameworks, messaging middleware, and servers for HTML5 multiplayer games. This article is a very elementary introduction to Node.js.

Overview of Blocking vs Non-Blocking

- **Blocking Code:** Code execution stops until a task completes, holding up other operations.
- **Non-Blocking Code:** Code execution continues while waiting for a task to complete, allowing other operations to proceed concurrently.

The steps below explain the difference between blocking code and non-blocking code models.

In the blocking code model, the steps are:

1. Read a file
2. Process the file
3. Print the result
4. Perform the next function

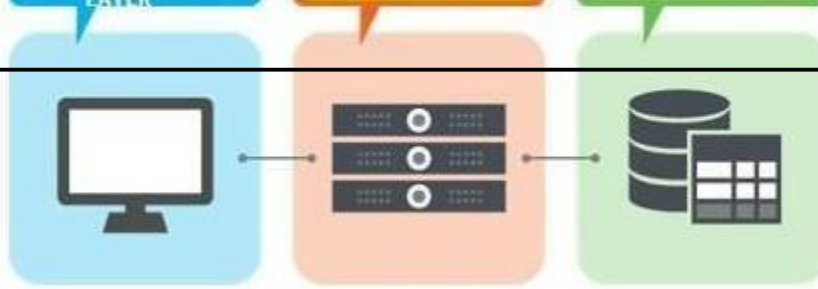
The steps in the non-blocking code model are:

1. Read a file
 - 1.1. When file reading is completed, process it
 - 1.1.1. When the processing is completed, print the result
2. Perform the next function

Feature	Blocking Code	Non-Blocking Code
Execution Model	Sequential, one task must complete before the next begins	Concurrent, tasks can be initiated and completed independently
Efficiency	Can be inefficient for I/O operations, leading to performance bottlenecks	More efficient, especially for I/O operations, as multiple tasks can proceed simultaneously
Complexity	Simpler to write and understand	Can be more complex due to the use of callbacks, promises, or async/await
Use Case	Suitable for tasks that require sequential execution	Suitable for tasks that involve I/O operations and can benefit from concurrency
Examples	Reading a file synchronously	Reading a file asynchronously using callbacks or promises

MULTI-TIER ARCHITECTURE

Multi-tier architecture provides a structured approach to designing and developing complex applications. It promotes separation of concerns, scalability, reusability, and security. By dividing the application into layers, developers can focus on specific areas of functionality and create applications that are easier to develop, maintain, and scale.



The three-tier architecture is a design pattern that separates applications into web layers (client layers), application layers, and database layers. Each layer has a specific responsibility and communicates with the other layers through well-defined interfaces. Since it's the most basic and popular design, this post will take a look at the three-tier architecture with simple code.

Web Layer

In the web layer, we're typically dealing with the front-end portion of our application, which includes the client (e.g., web browser) and the server-side code that handles incoming requests and sends back responses.

```
const http = require('http');
const hostname = 'localhost';
const port = 3000;

const app = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello World!');
});

app.listen(port, hostname, () => {
```

In the code above, we create a new HTTP server using the http module that's built into Node.js. We then listen for incoming requests on port 3000 and respond to each request with a plain text message of "Hello World!".

Application Layer

- ✓ The application layer is where we typically handle the business logic of our application. In a Node.js application, we might use an MVC architecture to separate the concerns of our application into different components.
- ✓ Define controller functions that encapsulate the application logic.
- ✓ Use services to encapsulate reusable business logic across multiple controllers.
- ✓ Implement middleware functions to handle common tasks like authentication or input validation. Let's use the Express Web framework to implement a simple MVC architecture in Node.js.

```
const express = require('express');
const app = express();
// Model - MySQL Database connection
const mysql = require('mysql');
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'username',
  password: 'password',
  database: 'mydb'
```

```
});  
// View - using EJS template engine  
app.set('view engine', 'ejs');  
// Controller - using Singleton pattern  
class UserController {  
  static getInstance() {  
    if (!UserController.instance) {  
      UserController.instance = new UserController();  
    }  
    return UserController.instance;  
  }  
  getUser(req, res) {  
    // get user data from database  
    connection.query('SELECT * FROM users WHERE id = ?', [req.params.id], (error, results) => {  
      if (error) {  
        console.log(error);  
        res.status(500).send('Error retrieving user data');  
      } else {  
        res.render('user', { user: results[0] });  
      }  
    });  
  }  
}  
// Routes  
const userController = UserController.getInstance();  
app.get('/user/:id', userController.getUser);  
// Start server  
app.listen(3000, () => {  
  console.log('Server started on port 3000');  
});
```

In the code above, we first create a MySQL database connection using the `mysql` module. We also configure our Express app to use the EJS template engine for rendering views.

Next, we create a `UserController` class that follows the Singleton pattern, which allows us to share a single instance of the controller across the application. In this example, the `getUser()` method is responsible for retrieving user data from the database and rendering a view using EJS. Finally, we define a route for the `/user/:id` URL and map it to the `getUser()` method of our `UserController`. When a request is made to this URL, the controller's `getUser()` method is called, which retrieves user data from the database and renders a view using EJS.

Database Layer

- ✓ Define models to represent data structures and interact with the database.
- ✓ Use ORM (Object-Relational Mapping) libraries like Sequelize or Mongoose to simplify database operations.
- ✓ Implement data access methods to perform CRUD (Create, Read, Update, Delete) operations on the database.

In the database layer, we're typically dealing with interactions with a database.

```

const mysql = require('mysql');
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'username',
  password: 'password',
  database: 'mydb'
});
connection.connect((err) => {
  if (err) throw err;
  console.log('Connected to MySQL database!');
  const createTableQuery = `CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(255),
    email VARCHAR(255) )`;
  connection.query(createTableQuery, (err, result) => {
    if (err) throw err;
    console.log('User table created successfully!');
  });
});

```

In the code above, we create a MySQL database connection using the mysql module. We then define a createTableQuery variable that contains the SQL query for creating a users table with id, name, and email columns.

Finally, we use the connection.query() method to execute the SQL query, which creates the users table in the database. If the query is successful, we log a message to the console indicating that the table was created successfully.

To improve their maintainability and flexibility, applications are often divided into several logical layers; a layer being an abstraction designed to satisfy a particular business need. Each layer is given a specific set of responsibilities, and can only access the one below it or at the same level (i.e. separation of concerns).

The Three-Tier Architecture is a powerful design pattern for building scalable and maintainable applications. By separating concerns into presentation, business logic, and data access layers, developers can create modular and flexible systems. When implemented using Node.js, each tier can be developed independently using frameworks and libraries that best suit its requirements. Whether you're building a small web application or a large-scale enterprise system, understanding and applying the principles of the Three-Tier Architecture can greatly improve the quality and maintainability of your codebase.

Node.js Architecture

Node.js is an extremely powerful JavaScript-based platform that's built on Google Chrome's JavaScript V8 Engine, used to develop I/O intensive web applications like video streaming sites, single-page applications, online chat applications, and other web apps.

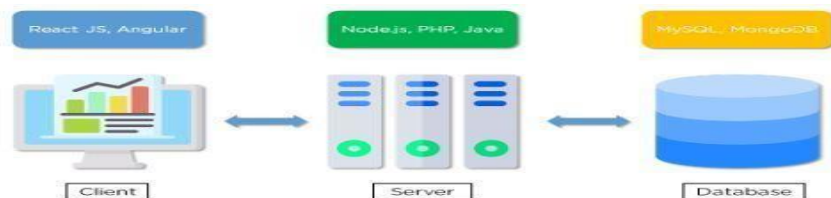
Node.js is used by large, established companies and newly-minted startups alike. Open-source and completely free, the platform is used by thousands of developers around the world. It brings plenty of advantages to the table, making it a better choice than other server-side platforms like Java or PHP in many cases.

Node.js architecture:

- ✓ Node.js server architecture
- ✓ Parts of the Node.js architecture
- ✓ Workflow of Node.js architecture

Advantages of Node.js architectureWeb Applications

A web application, as you may already know, is a program that runs on a server and is rendered by a client browser, using the internet to access all the resources of that application. It usually can be easily broken down into three parts:



Client

The user interacts with the front-end part of a web application. The front-end is usually developed using languages like HTML and CSS styles, along with extensive usage of JavaScript-based frameworks like ReactJS and Angular, which help with application design.

Server

The server is responsible for taking the client requests, performing the required tasks, and sending responses back to the clients. It acts as a middleware between the front-end and stored

data to enable operations on the data by a client. Node.js, PHP, and Java are the most popular technologies in use to develop and maintain a web server.

Database

The database stores the data for a web application. The data can be created, updated, and deleted whenever the client requests. MySQL and MongoDB are among the most popular databases used to store data for web applications.

Node.js Server Architecture

Node.js uses the “Single Threaded Event Loop” architecture to handle multiple concurrent clients. Node.js Processing Model is based on the JavaScript event-based model along with the JavaScript call-back mechanism.

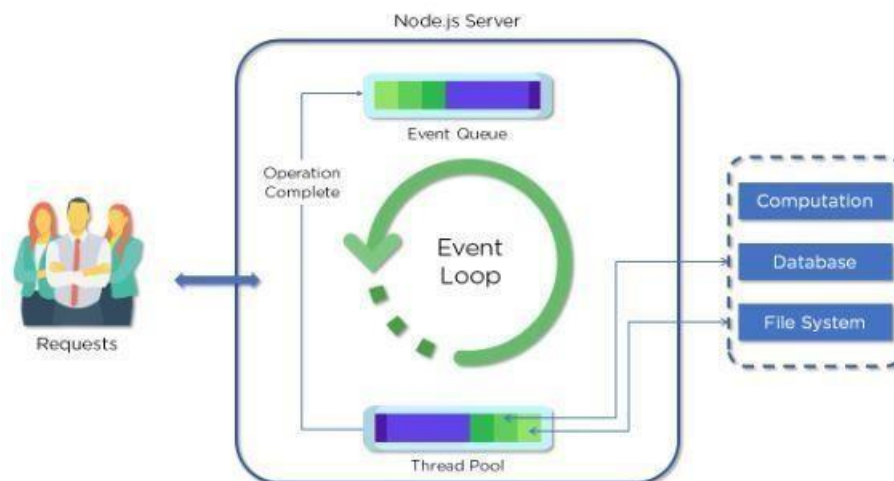


Fig: Node.js architecture

Now let's understand each part of the Node.js architecture and the workflow of a web server developed using Node.js.

Parts of the Node.js Architecture:

- Requests

Incoming requests can be blocking (complex) or non-blocking (simple), depending upon the tasks that a user wants to perform in a web application

- Node.js Server

Node.js server is a server-side platform that takes requests from users, processes those requests, and returns responses to the corresponding users

- Event Queue

Event Queue in a Node.js server stores incoming client requests and passes those requests one-by-one into the Event Loop

- Thread Pool

Thread pool consists of all the threads available for carrying out some tasks that might be required to fulfill client requests

- Event Loop

Event Loop indefinitely receives requests and processes them, and then returns the responses to corresponding clients

- External Resources

External resources are required to deal with blocking client requests. These resources can be for computation, data storage, etc.

The Workflow of Node.js Architecture:

A web server developed using Node.js typically has a workflow that is quite similar to the diagram illustrated below. Let's explore this flow of operations in detail.

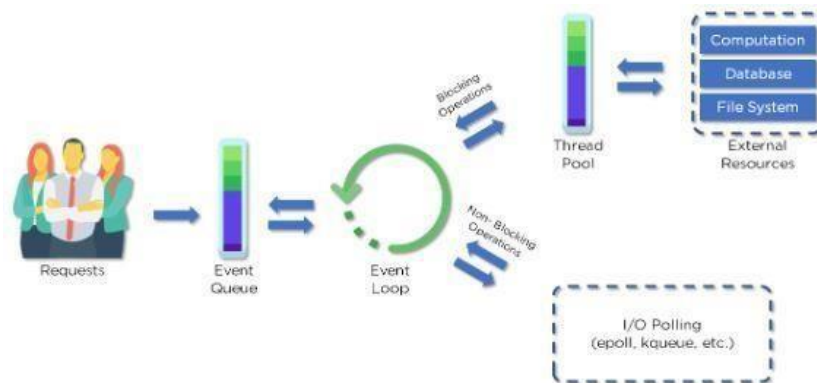


Fig: Node.js Architecture Workflow

- Clients send requests to the webserver to interact with the web application. Requests can be non-blocking or blocking:

-Querying for data

-Deleting data

-Updating the data

- Node.js retrieves the incoming requests and adds those requests to the Event Queue
- The requests are then passed one-by-one through the Event Loop. It checks if the requests are simple enough to not require any external resources
- Event Loop processes simple requests (non-blocking operations), such as I/O Polling, and returns the responses to the corresponding clients

A single thread from the Thread Pool is assigned to a single complex request. This thread is responsible for completing a particular blocking request by accessing the external resources, such as compute, database, file system, etc.

Once, the task is carried out completely, the response is sent to the Event Loop that in turn sends that response back to the Client

Advantages of Node.js Architecture

Node.js Architecture comes with several advantages that give the server-side platform a distinct upper-hand when compared to other server-side languages:

- ✓ Handling multiple concurrent client requests is fast and easy

With the use of Event Queue and Thread Pool, the Node.js server enables efficient handling of a large number of incoming requests.

- ✓ No need for creating multiple threads

Event Loop handles all requests one-by-one, so there is no need to create multiple threads. Instead, a single thread is sufficient to handle a blocking incoming request.

- ✓ Requires fewer resources and memory

Node.js server, most of the time, requires fewer resources and memory due to the way it handles the incoming requests. Since the requests are processed one at a time, the overall process becomes less taxing on the memory.

All of these advantages contribute to making the servers developed using Node.js much faster and responsive when compared to those developed using other server development technologies.

NPM (Node Package Manager)

NPM stands for Node Package Manager. It is the Node.js default package manager and is completely written in Javascript. NPM is a command-line client for Node.js that manages all of the dependencies and modules. With the installation of Node.js, it is added to the system. Any required packages and modules in node projects are installed using NPM. A package contains all

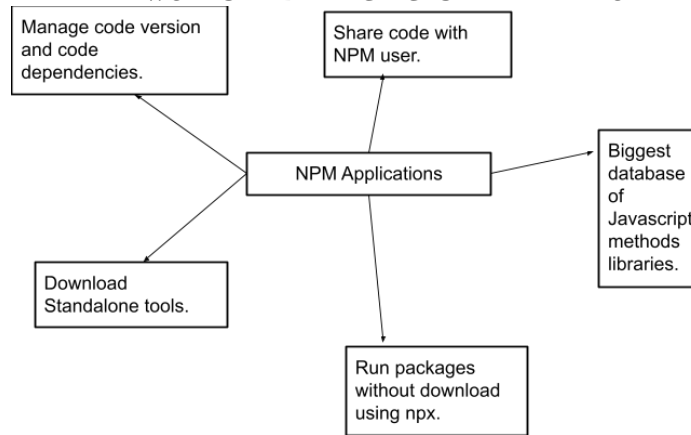
of the files required for a module, and modules are JavaScript libraries that can be included in a Node project based on the project's requirements. It includes a large number of libraries that serve as excellent tools for Node.js developers, speeding up the entire application development process. NPM can use the package.json file to install all of a project's dependencies. It has the ability to upgrade and remove packages. Each dependency may specify a range of valid versions using the semantic versioning scheme in the package.json file, allowing developers to auto-update their packages while avoiding unwanted breaking changes.

Some basic commands:-

- **npm i <packageName>** :- installs a local package.
- **npm i -g <packageName>** :- installs a global package.
- **npm un <packageName>** :- uninstalls a local package.
- **npm up** :- updates the packages.
- **npm t**:- runs the tests.
- **npm ls** :- lists all the installed modules.
- **npm ll** :- prints additional package information while listing modules.

Uses of NPM:-

- It facilitates the integration of pre-built packages into our project.
- It aids in the download of a number of standalone tools that can be used right away.
- NPM is frequently used by developers to share their code with other NPM users all over the world.
- It helps in managing and maintaining various versions of codes and their dependencies.
- NPM automatically updates the application when the underlying codes are updated.
- It functions as a community where you can connect with other developers who are working on similar projects or tasks.



Some terminologies before getting into the working of NPM:-

❖ Package:-

A package must contain a package.json file in order to be published to the npm registry. Packages can be unscoped or scoped to a user or organization, and scoped packages can be private or public.

A package is any of the following:

- A folder containing a program described by a package.json file.
- A zipped tarball containing (a).
- A URL that resolves to (b).
- A <name>@<version> that is published on the registry with (c).
- A <name>@<tag> that points to (d).
- A <name> that has a latest tag satisfying (e).
- A git url that, when cloned, results in (a).

❖ Module :-

From the official documentation, a module is any file or directory in the node_modules directory that can be loaded by the Node.js require() function. To be loaded by the Node.js require() function, a module must be one of the following:

1. A folder with a package.json file containing a "main" field.
2. A JavaScript file.

Since modules are not required to have a package.json file, not all modules are packages. Only modules that have a package.json file are packages. In the context of a Node program, the module is also the thing that was loaded from a file.

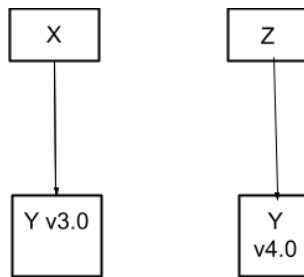
For example, in the following program:

```
var req = require('request')
```

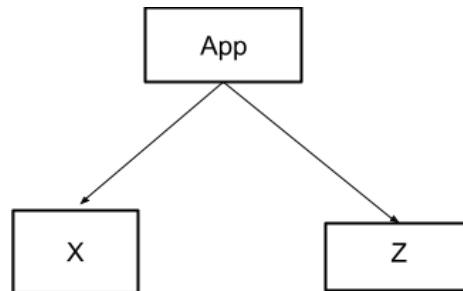
We can say that the variable 'req' refers to the 'request' module.

❖ Dependency hell:-

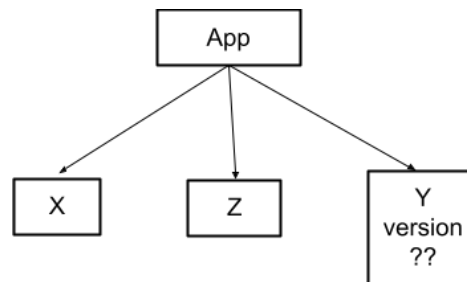
Let us assume that there are three modules : X, Y and Z. X requires Y at v3.0, and Z also requires Y, but at v4.0. We can visualize this like so:



Let us assume that we have an application that requires both module X and module Z.



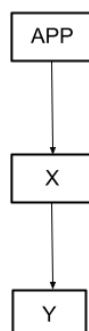
Now, there will be a confusion as to which version of B is to be included. This is called **Dependency Hell**.



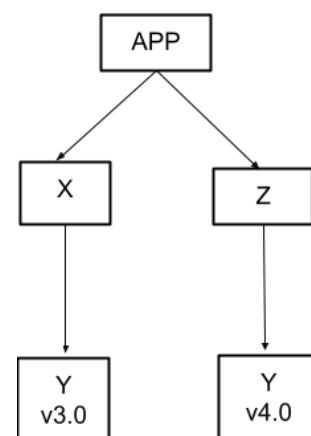
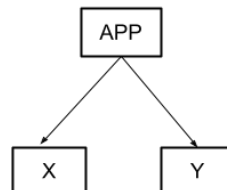
Working of NPM version 2:-

In case of Dependency Hell, as discussed above, instead of attempting to resolve module Y to a single version, npm version 2 puts both versions of module Y into the tree, each version nested under the module that requires it.

npm
version
2



npm
version
3



Working of NPM version 3:-

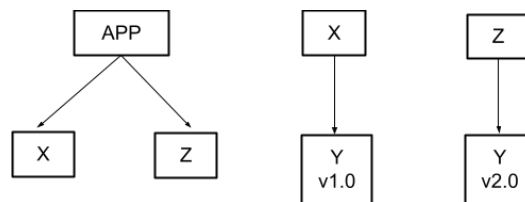
In npm3, dependencies are resolved in a different way than in npm2. Although npm2 installs all dependencies in a nested fashion, npm3 tries to avoid the deep trees and redundancy that such nesting causes. npm3 tries to accomplish this by flatly installing secondary dependencies (dependencies of dependencies) in the same directory as the primary dependency that requires them.

The following are the most significant differences:

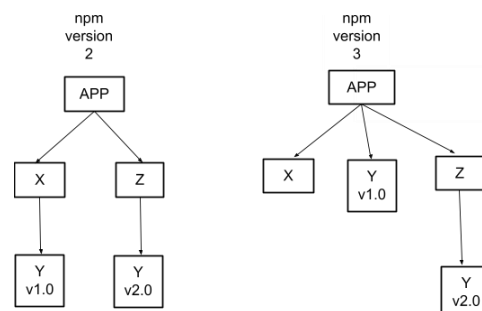
- ❖ The type (primary, secondary, etc.) of a dependency is no longer determined by its location in the directory structure.
- ❖ The order in which things are installed will change the node modules directory tree structure, since dependency resolution is dependent on installation order.

Let us assume we have a module X. X requires Y. Now let's create an application that requires module X. On npm install, npm version 3 will install both module X and its dependency, module Y, inside the /node_modules directory, flat. In npm version 2 this would have happened in a nested way.

Let us suppose that we want to require another module, Z. Z requires Y, but at another version than X.



Since Y v1.0 is already a top-level dependency, we cannot install Y v2.0 as a top level dependency. npm v3 handles this by behaving in a similar fashion to npm v2 and nesting the new, different, module Y version dependency under the module that requires it, that is, module Z.

**Installation of NPM:-**

To install NPM, it is required to install Node.js as NPM gets installed with Node.js automatically. Versions of npm installed on the system can be checked using the following command:

npm -v.

If the version is not the latest one, we can install it using the command:

```
npm npm@latest -g
```

Creating a Node Project:-

To create a new node project, we use `npm init` in the folder in which we want to create the project. A number of questions like name, license, scripts, description, author, keywords, version and main file will be asked. After the project is created, a `package.json` file will be created in the project folder which confirms that the project has been initialized.

Installation of Packages and Modules:-

Following the creation of the project, the next step is to add the packages and modules that will be used in the Node Project. Use the following code to install packages and modules in the project:

```
npm install package_name
```

Example:

To install the mongoose package in our project, we use the following command :

```
npm install mongoose
```

We can add an extra `-g` tag to the package installation syntax to make it global, that is, accessible by all projects in the system.

Controlling the versions of the packages to be installed:-

- In the `package.json` file, specify the complete and exact version to install a package of a particular version.
- To install any version above a given version, mention it like :

“`packageName`” : “`^<version Number>`” in `package.json` file.

The caret symbol (^) tells the npm to find a version greater than `<version Number>` and install it.

- Mention “`*`” in front of the dependency or “`latest`” to install the most recent version of the package. This will locate and install the most recent stable version of the module.

Controlling where the packages and the modules get installed:-

We can add the `–save` flag to install a package while also saving it to the `package.json` file.

Because the `–save` flag is set by default in the `npm install` command, it is equivalent to `npm install package name`.

–save-prod: Installing this package causes it to appear in Dependencies, which is also the default.

–save-dev: This package will appear in devDependencies and will only be used in development mode if you use it.

Simply type `npm install` in the terminal if there is already a `package.json` file with all the packages listed as dependencies. NPM will examine the `package.json` file and install all of the dependencies in the order in which they are listed in the file. When a Node project is forked and cloned, this command is usually used.

NPM installs dependencies in local mode (by default), to the node modules directory of the Node application folder.

Use the npm ls command to see all the locally installed modules.

Uninstalling Packages:-

To uninstall packages using npm, type the command:

```
npm uninstall
```

To uninstall global packages, type the command:

```
npm uninstall package_name -g
```

Some Node.Js Frameworks: -

- Express.js - Express for Everyone
- Koa.js - Next Generation Node.js
- Framework Meteor.js - One Application, One Language
- Socket.io - Chat Apps Made Easy with Socket.io
- Nest.js - A Nestling of Code

DEVELOPMENT ENVIRONMENT

One way of doing this is to install everything into \$HOME/local/\$PACKAGE. Here is how to install
./configure --prefix=\$HOME/local/node-v0.4.5 && make install

To have my paths automatically set I put this inside my \$HOME/.zshrc:

```
PATH="$HOME/local/bin:/opt/local/bin:/usr/bin:/sbin:/bin"
```

```
LD_LIBRARY_PATH="/opt/local/lib:/usr/local/lib:/usr/lib"
```

```
for i in $HOME/local/*; do
```

```
[ -d $i/bin ] && PATH="$i/bin:${PATH}"
```

```
[ -d $i/sbin ] && PATH="$i/sbin:${PATH}"
```

```
[ -d $i/include ] && CPATH="$i/include:${CPATH}"
```

```
[ -d $i/lib ] && LD_LIBRARY_PATH="$i/lib:${LD_LIBRARY_PATH}"
```

```
[ -d $i/lib/pkgconfig ] &&
```

```
PKG_CONFIG_PATH="$i/lib/pkgconfig:${PKG_CONFIG_PATH}"
```

```
[ -d $i/share/man ] && MANPATH="$i/share/man:${MANPATH}"done
```

Node is under sufficiently rapid development that *everyone* should be compiling it themselves. A corollary of this is that npm (which should be installed alongside Node) does not require root to install packages.

CPAN and RubyGems have blurred the lines between development tools and system package managers. With npm we wish to draw a clear line: it is not a system package manager. It is not for installing firefox or ffmpeg or OpenSSL; it is for rapidly downloading, building, and setting up Node packages. npm is a *development* tool. When a program written in Node becomes sufficiently mature it should be distributed as a tarball, .deb, .rpm, or other package system. It should not be distributed to end users with npm.

How to Set Up Your Local Node.js Development Environment Using Docker

Docker is the de facto toolset for building modern applications and setting up a CI/CD pipeline – helping you build, ship, and run your applications in containers on-prem and in the cloud.

Whether you're running on simple compute instances such as AWS EC2 or something fancier like a hosted Kubernetes service, Docker's toolset is your new BFF.

But what about your local Node.js development environment? Setting up local dev environments while also juggling the hurdles of onboarding can be frustrating, to say the least.

How to set up a local Node.js dev environment — Part 1

In this tutorial, we'll walk through setting up a local Node.js development environment for a relatively complex application that uses React for its front end, Node and Express for a couple of micro-services, and MongoDB for our datastore. We'll use Docker to build our images and Docker Compose to make everything a whole lot easier.

If you have any questions, comments, or just want to connect. You can reach me in our Community Slack or on Twitter at @rumpl.

Let's get started.

Prerequisites

You will need to:

- Docker installed on your development machine. You can download and install Docker Desktop.
- Sign-up for a Docker ID through Docker Hub.
- Git installed on your development machine.
- An IDE or text editor to use for editing files. I would recommend VSCode.

Step 1: Fork the Code Repository

The first thing we want to do is download the code to our local development machine. Let's do this using the following git command:

```
git clone https://github.com/rumpl/memphis.git
```

Now that we have the code local, let's take a look at the project structure. Open the code in your favorite IDE and expand the root level directories. You'll see the following file structure.

```
|— docker-compose.yml
|— notes-service
```

- reading-list-service
- users-service
- yoda-ui

The application is made up of a couple of simple microservices and a front-end written in React.js. It also uses MongoDB as its datastore.

Typically at this point, we would start a local version of MongoDB or look through the project to find where our applications will be looking for MongoDB. Then, we would start each of our microservices independently and start the UI in hopes that the default configuration works.

However, this can be very complicated and frustrating. Especially if our microservices are using different versions of Node.js and configured differently.

Instead, let's walk through making this process easier by dockerizing our application and putting our database into a container.

Step 2: Dockerize your applications

Docker is a great way to provide consistent development environments. It will allow us to run each of our services and UI in a container. We'll also set up things so that we can develop locally and start our dependencies with one docker command.

The first thing we want to do is dockerize each of our applications. Let's start with the microservices because they are all written in Node.js, and we'll be able to use the same Dockerfile.

Creating Dockerfiles

Create a Dockerfile in the notes-services directory and add the following commands.

```
FROM node:18.7.0

WORKDIR /code

COPY package.json package.json
COPY package-lock.json package-lock.json

RUN npm install

COPY . .

CMD [ "node", "server.js" ]
```

This is a very basic Dockerfile to use with Node.js.

Building Docker Images

Now that we've created our Dockerfile, let's build our image. Make sure you're still located in the notes-services directory and run the following command:

```
cd notes-service
```

```
docker build -t notes-service .
```

```
> docker build -t notes-service .

[+] Building 0.6s (11/11) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 205B 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 53B 0.0s
=> [internal] load metadata for docker.io/library/node:18.7.0 0.6s
=> [1/6] FROM docker.io/library/node:18.7.0@sha256:a96207e01265169cb1ecbbb4b6af30d7 0.0s
=> => resolve docker.io/library/node:18.7.0@sha256:a96207e01265169cb1ecbbb4b6af30d7 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 367B 0.0s
=> CACHED [2/6] WORKDIR /code 0.0s
=> CACHED [3/6] COPY package.json package.json 0.0s
=> CACHED [4/6] COPY package-lock.json package-lock.json 0.0s
=> CACHED [5/6] RUN npm install 0.0s
=> CACHED [6/6] COPY . . 0.0s
=> exporting to image 0.0s
=> => exporting layers 0.0s
=> => exporting manifest sha256:de55aa02e1af68a9d46574ae6d6139a0443446b245d2a9d4f5a 0.0s
=> => exporting config sha256:7487eb2e8d98da1b6be9a5e9bb7e55b98f0b170ccf5c159dd0a97 0.0s
=> => naming to docker.io/library/notes-service:latest 0.0s
=> => unpacking to docker.io/library/notes-service:latest 0.0s
```

Now that we have our image built, let's run it as a container and test that it's working. `docker run -rm -p 8081:8081 --name notes notes-service`

```
> docker run --rm -p 8081:8081 --name notes notes-service

config.database.connectionstring undefined
Error: Connection String Required. Please provide a mongodb connection string. See
https://docs.mongodb.com/manual/reference/connection-string/ for more details.
    at Object.connect (/code/node_modules/ronin-database/lib/index.js:8:9)
    at main (/code/server.js:12:20)
    at Object.<anonymous> (/code/server.js:40:1)
    at Module._compile (node:internal/modules/cjs/loader:1120:14)
    at Module._extensions..js (node:internal/modules/cjs/loader:1174:10)
    at Module.load (node:internal/modules/cjs/loader:998:32)
    at Module._load (node:internal/modules/cjs/loader:839:12)
    at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:81:12)
    at node:internal/main/run_main_module:17:47
```

From this error, we can see we're having trouble connecting to the mongodb. Two things are broken at this point:

1. We didn't provide a connection string to the application.
2. We don't have MongoDB running locally.

To resolve this, we could provide a connection string to a shared instance of our database, but we want to be able to manage our database locally and not have to worry about messing up our colleagues' data they might be using to develop.

Step 3: Run MongoDB in a localized container

Instead of downloading MongoDB, installing, configuring, and then running the Mongo database service. We can use the Docker Official Image for MongoDB and run it in a container.

Before we run MongoDB in a container, we want to create a couple of volumes that Docker can manage to store our persistent data and configuration. I like to use the managed volumes that Docker provides instead of using bind mounts. You can read all about volumes in our documentation.

Creating volumes for Docker

To create our volumes, we'll create one for the data and one for the configuration of MongoDB.

`docker volume create mongodb`

`docker volume create mongodb_config` Creating a user-defined bridge network

Now we'll create a network that our application and database will use to talk with each other. The network is called a user-defined bridge network and gives us a nice DNS lookup service that we can use when creating our connection string.

Now, we can run MongoDB in a container and attach it to the volumes and network we created above. Docker will pull the image from Hub and run it for you locally.

```
docker run  
-it --rm  
-d -v mongodb:/data/db  
-v mongodb_config:/data/configdb  
-p27017:27017  
--network mongodb --name mongodb mongo
```

Step 4: Set your environment variables

Now that we have a running MongoDB, we also need to set a couple of environment variables so our application knows what port to listen on and what connection string to use to access the database. We'll do this right in the docker run command.

```
docker run \  
-it --rm -d \  
--network mongodb \  
--name notes \  
-p 8081:8081 \  
-e SERVER_PORT=8081 \  
-e SERVER_PORT=8081 \  
-e DATABASE_CONNECTIONSTRING=mongodb://mongodb:27017/yoda_notes \notes-  
service
```

Step 5: Test your database connection

Let's test that our application is connected to the database and is able to add a note. `curl --request POST \`

```
--url http://localhost:8081/services/m/notes \  

```

```
--header 'content-type: application/json' \  

```

```
--data '{  

```

```
"name": "this is a note",  

```

```
"text": "this is a note that I wanted to take while I was working on writing a blog post.", "owner":  

```

```
"peter"  

```



```
}'
```

You should receive the following JSON back from our service.

```
{ "code": "success", "payload": { "_id": "5efd0a1552cd422b59d4f994", "name": "this is a note", "text": "this is a note that I wanted to take while I was working on writing a blog post.", "owner": "peter", "createDate": "2020-07-01T22:11:33.256Z" } }
```

Once we are done testing, run ‘docker stop notes mongodb’ to stop the containers.

Awesome! We’ve completed the first steps in Dockerizing our local development environment for Node.js. In Part II, we’ll take a look at how we can use Docker Compose to simplify the process we just went through.

How to set up a local Node.js dev environment — Part 2

In Part I, we took a look at creating Docker images and running containers for Node.js applications. We also took a look at setting up a database in a container and how volumes and networks play a part in setting up your local development environment.

In Part II, we’ll take a look at creating and running a development image where we can compile, add modules and debug our application all inside of a container. This helps speed up the developer setup time when moving to a new application or project. In this case, our image should have Node.js installed as well as NPM or YARN.

We’ll also take a quick look at using Docker Compose to help streamline the processes of setting up and running a full microservices application locally on your development machine.

Let’s create a development image we can use to run our Node.js application.

Step 1: Develop your Dockerfile

Create a local directory on your development machine that we can use as a working directory to save our Dockerfile and any other files that we’ll need for our development image.

```
$ mkdir -p ~/projects/dev-image
```

Create a Dockerfile in this folder and add the following commands. FROM node:18.7.0

```
RUN apt-get update && apt-get install -y \nano \
```

```
Vim
```

We start off by using the node:18.7.0 official image. I’ve found that this image is fine for creating a development image. I like to add a couple of text editors to the image in case I want to quickly edit a file while inside the container.

We did not add an ENTRYPOINT or CMD to the Dockerfile because we will rely on the base image’s ENTRYPOINT, and we will override the CMD when we start the image.

Step 2: Build your Docker image

Let's build our image.

\$ docker build -t node-dev-image . And now we can run it.

\$ docker run -it --rm --name dev -v \$(pwd):/code node-dev-image bash

You will be presented with a bash command prompt. Now, inside the container, we can create a JavaScript file and run it with Node.js.

Step 3: Test your image

Run the following commands to test our image.

\$ node -e 'console.log("hello from inside our container")' hello from inside our container

If all goes well, we have a working development image. We can now do everything that we would do in our normal bash terminal.

If you run the above Docker command inside of the notes-service directory, then you will have access to the code inside of the container. You can start the notes-service by simply navigating to the /code directory and running npm run start.

Step 4: Use Compose to Develop locally

The notes-service project uses MongoDB as its data store. If you remember from Part I, we had to start the Mongo container manually and connect it to the same network as our notes-service. We also had to create a couple of volumes so we could persist our data across restarts of our application and MongoDB.

Instead, we'll create a Compose file to start our notes-service and the MongoDB with one command. We'll also set up the Compose file to start the notes-service in debug mode. This way, we can connect a debugger to the running node process.

Open the notes-service in your favorite IDE or text editor and create a new file named docker-compose.dev.yml. Type the py and paste the below commands into the file.

```
1  services:
2  notes:
3  build:
4    context: .ports:
5    - 8080:8080
6    - 9229:9229
7  environment:
8    SERVER_PORT=8080
9    - DATABASE_CONNECTIONSTRING=mongodb://mongo:27017/notesvolumes:
10     ./:/code
11  command: npm run debug
12  mongo:
13  image: mongo:4.2.8ports:
14  - 27017:27017
15  volumes:
16    - mongodb:/data/db
17    - mongodb_config:/data/configdbvolumes:
18  mongodb:
19  Mongodb_config:
```

This compose file is super convenient because now we don't have to type all the parameters to pass to the `docker run` command. We can declaratively do that in the compose file.

We are exposing port 9229 so that we can attach a debugger. We are also mapping our local source code into the running container so that we can make changes in our text editor and have those changes picked up in the container.

One other really cool feature of using the compose file is that we have service resolution setup to use the service names. As a result, we are now able to use "mongo" in our connection string. We use "mongo" because that is what we have named our mongo service in the compose file.

Let's start our application and confirm that it is running properly.

```
$ docker compose -f docker-compose.dev.yml up --build
```

We pass the "--build" flag so Docker will compile our image and then start it. If all goes well, you should see the logs from the notes and mongo services

```

docker compose -f docker-compose.dev.yml up --build

ugh 2
notes-service-mongo-1 | 2022-08-26T12:45:16.701+0000 I STORAGE [initandlisten] WiredTiger message [1661517916:701963][1:0xffffa67ee450], txn-recover: Recovering log 2 thro
ugh 2
notes-service-mongo-1 | 2022-08-26T12:45:16.734+0000 I STORAGE [initandlisten] WiredTiger message [1661517916:734615][1:0xffffa67ee450], txn-recover: Set global recovery t
imestamp: (0, 0)
notes-service-mongo-1 | 2022-08-26T12:45:16.744+0000 I RECOVERY [initandlisten] WiredTiger recoveryTimestamp. Ts: Timestamp(0, 0)
notes-service-mongo-1 | 2022-08-26T12:45:16.747+0000 I STORAGE [initandlisten] Timestamp monitor starting
notes-service-mongo-1 | 2022-08-26T12:45:16.749+0000 I CONTROL [initandlisten]
notes-service-mongo-1 | 2022-08-26T12:45:16.749+0000 I CONTROL [initandlisten] ** WARNING: Access control is not enabled for the database.
notes-service-mongo-1 | 2022-08-26T12:45:16.749+0000 I CONTROL [initandlisten] ** Read and write access to data and configuration is unrestricted.
notes-service-mongo-1 | 2022-08-26T12:45:16.749+0000 I CONTROL [initandlisten]
notes-service-mongo-1 | 2022-08-26T12:45:16.752+0000 I SHARDING [initandlisten] Marking collection local.system.replset as collection version: <unsharded>
notes-service-mongo-1 | 2022-08-26T12:45:16.754+0000 I STORAGE [initandlisten] Flow Control is enabled on this deployment.
notes-service-mongo-1 | 2022-08-26T12:45:16.754+0000 I SHARDING [initandlisten] Marking collection admin.system.roles as collection version: <unsharded>
notes-service-mongo-1 | 2022-08-26T12:45:16.754+0000 I SHARDING [initandlisten] Marking collection admin.system.version as collection version: <unsharded>
notes-service-mongo-1 | 2022-08-26T12:45:16.755+0000 I SHARDING [initandlisten] Marking collection local.startup_log as collection version: <unsharded>
notes-service-mongo-1 | 2022-08-26T12:45:16.755+0000 I FTDC [initandlisten] Initializing full-time diagnostic data capture with directory '/data/db/diagnostic.data'
notes-service-mongo-1 | 2022-08-26T12:45:16.757+0000 I SHARDING [LogicalSessionCacheRefresh] Marking collection config.system.sessions as collection version: <unsharded>
notes-service-mongo-1 | 2022-08-26T12:45:16.757+0000 I SHARDING [LogicalSessionCacheRefresh] Marking collection config.transactions as collection version: <unsharded>
notes-service-mongo-1 | 2022-08-26T12:45:16.758+0000 I NETWORK [listener] Listening on /tmp/mongod-27017.sock
notes-service-mongo-1 | 2022-08-26T12:45:16.758+0000 I NETWORK [listener] Listening on 0.0.0.0
notes-service-mongo-1 | 2022-08-26T12:45:16.758+0000 I NETWORK [listener] waiting for connections on port 27017
notes-service-notes-1 |
> notes-service@1.0.0 debug
notes-service-notes-1 | > nodemon --inspect=0.0.0:9229 server.js
notes-service-notes-1 |
notes-service-mongo-1 | 2022-08-26T12:45:17.000+0000 I SHARDING [ftdc] Marking collection local.oplog.rs as collection version: <unsharded>
notes-service-notes-1 | [nodemon] 2.0.3
notes-service-notes-1 | [nodemon] to restart at any time, enter 'rs'
notes-service-notes-1 | [nodemon] watching path(s): *.*
notes-service-notes-1 | [nodemon] watching extensions: js,mjs,json
notes-service-notes-1 | [nodemon] starting 'node --inspect=0.0.0:9229 server.js'
notes-service-notes-1 | Debugger listening on ws://0.0.0.0:9229/70b3b956-626f-48f2-9b2c-c7eb5fald99e
notes-service-notes-1 | For help, see: https://nodejs.org/en/docs/inspector
notes-service-notes-1 | config.database.connectionstring mongod://mongo:27017/notes
notes-service-mongo-1 | 2022-08-26T12:45:20.761+0000 I NETWORK [listener] connection accepted from 172.18.0.3:52704 #1 (1 connection now open)
notes-service-mongo-1 | 2022-08-26T12:45:20.765+0000 I NETWORK [conn1] received client metadata from 172.18.0.3:52704 conn1: { driver: { name: "nodejs", version: "3.5.9" }
, os: { type: "Linux", name: "Linux", architecture: "arm64", version: "5.10.124-linuxkit" }, platform: "Node.js v18.7.0, LE (unified)" }
notes-service-notes-1 | Listening on port 8080

```

Now let's test our API endpoint. Run the following curl command:

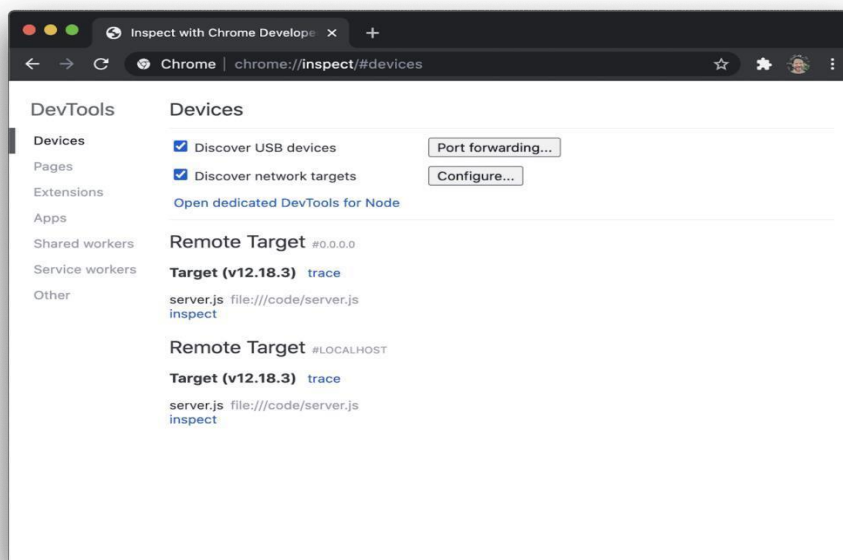
\$ curl --request GET --url http://localhost:8080/services/m/notes You should receive the following response:

```
{"code": "success", "meta": {"total": 0, "count": 0}, "payload": []}
```

Step 5: Connect to a Debugger

We'll use the debugger that comes with the Chrome browser. Open Chrome on your machine, and then type the following into the address bar. The following screen will open.

About:inspect



Click the “Open dedicated DevTools for Node” link. This will open the DevTools that are connected to the running Node.js process inside our container.

Let’s change the source code and then set a breakpoint.

Add the following code to the server.js file on line 19 and save the file.

```
1 rver.use( '/foo', (req, res) => {
2
3 return res.json({ "foo": "bar" })
})
```

If you take a look at the terminal where our compose application is running, you’ll see that nodemon noticed the changes and reloaded our application.

```
docker compose -f docker-compose.dev.yml up --build
notes-service-mongo-1 | 2022-08-26T12:45:16.754+0000 I STORAGE [initandlisten] Flow Control is enabled on this deployment.
notes-service-mongo-1 | 2022-08-26T12:45:16.754+0000 I SHARDING [initandlisten] Marking collection admin.system.roles as collection version: <unsharded>
notes-service-mongo-1 | 2022-08-26T12:45:16.754+0000 I SHARDING [initandlisten] Marking collection admin.system.version as collection version: <unsharded>
notes-service-mongo-1 | 2022-08-26T12:45:16.755+0000 I SHARDING [initandlisten] Marking collection local.startup_log as collection version: <unsharded>
notes-service-mongo-1 | 2022-08-26T12:45:16.755+0000 I FTDC [initandlisten] Initializing full-time diagnostic data capture with directory '/data/db/diagnostic.data'
notes-service-mongo-1 | 2022-08-26T12:45:16.757+0000 I SHARDING [LogicalSessionCacheRefresh] Marking collection config.system.sessions as collection version: <unsharded>
notes-service-mongo-1 | 2022-08-26T12:45:16.758+0000 I NETWORK [listener] Listening on /tmp/mongodb-27017.sock
notes-service-mongo-1 | 2022-08-26T12:45:16.758+0000 I NETWORK [listener] Listening on 0.0.0.0
notes-service-mongo-1 | 2022-08-26T12:45:16.758+0000 I NETWORK [listener] waiting for connections on port 27017
notes-service-notes-1 | > notes-service@1.0.0 debug
notes-service-notes-1 | > nodemon --inspect=0.0.0.0:9229 server.js
notes-service-mongo-1 | 2022-08-26T12:45:17.000+0000 I SHARDING [ftdc] Marking collection local.oplog.rs as collection version: <unsharded>
notes-service-notes-1 | [nodemon] 2.0.3
notes-service-notes-1 | [nodemon] to restart at any time, enter 'rs'
notes-service-notes-1 | [nodemon] watching path(s): *.*
notes-service-notes-1 | [nodemon] watching extensions: js,mjs,json
notes-service-notes-1 | [nodemon] starting 'node --inspect=0.0.0.0:9229 server.js'
notes-service-notes-1 | Debugger listening on ws://0.0.0.0:9229/70b3b956-626f-48f2-9b2c-c7eb5fald99e
notes-service-notes-1 | For help, see: https://nodejs.org/en/docs/inspector
notes-service-notes-1 | config.database.connectionstring mongodb://mongo:27017/notes
notes-service-mongo-1 | 2022-08-26T12:45:20.761+0000 I NETWORK [listener] connection accepted from 172.18.0.3:52704 #1 (1 connection now open)
notes-service-notes-1 | 2022-08-26T12:45:20.765+0000 I NETWORK [conn1] received client metadata from 172.18.0.3:52704 conn1: { driver: { name: "nodejs", version: "3.5.9" },
notes-service-mongo-1 |   os: { type: "Linux",
notes-service-notes-1 |     name: "Linux", architecture: "arm64", version: "5.10.124-linuxkit" }, platform: "Node.js v18.7.0, LE (unified)" }
notes-service-notes-1 |   Listening on port 8080
notes-service-notes-1 | Debugger attached.
notes-service-notes-1 | [nodemon] restarting due to changes...
notes-service-mongo-1 | 2022-08-26T12:46:47.670+0000 I NETWORK [conn1] end connection 172.18.0.3:52704 (0 connections now open)
notes-service-notes-1 | [nodemon] starting 'node --inspect=0.0.0.0:9229 server.js'
notes-service-notes-1 | Debugger listening on ws://0.0.0.0:9229/5905c232-e51a-4f0b-b322-44cd78151d16
notes-service-notes-1 | For help, see: https://nodejs.org/en/docs/inspector
notes-service-notes-1 | Debugger attached.
notes-service-mongo-1 | 2022-08-26T12:46:50.307+0000 I NETWORK [listener] connection accepted from 172.18.0.3:33010 #2 (1 connection now open)
notes-service-mongo-1 | 2022-08-26T12:46:50.311+0000 I NETWORK [conn2] received client metadata from 172.18.0.3:33010 conn2: { driver: { name: "nodejs", version: "3.5.9" },
notes-service-mongo-1 |   os: { type: "Linux",
notes-service-notes-1 |     name: "Linux", architecture: "arm64", version: "5.10.124-linuxkit" }, platform: "Node.js v18.7.0, LE (unified)" }
notes-service-notes-1 |   Listening on port 8080
```

Conclusion

In this article, we completed the first steps in Dockerizing our local development environment for Node.js. Then, we took things a step further and created a general development image that can be used like our normal command line. We also set up our compose file to map our source code into the running container and exposed the debugging port.

Node.js - RESTful API

REST stands for Representational State Transfer. REST is web standards based architecture and uses HTTP Protocol. It revolves around resource where every component is a resource and a resource is accessed by a common interface using HTTP standard methods. REST was first introduced by Roy Fielding in 2000.

A REST Server simply provides access to resources and REST client accesses and modifies the resources using HTTP protocol. Here each resource is identified by URIs/ global IDs. REST uses various representations to represent a resource like text, JSON, XML but JSON is the most

popular one. HTTP methods

Following four HTTP methods are commonly used in REST based architecture.

- **GET** – This is used to provide a read only access to a resource.
- **PUT** – This is used to create a new resource.
- **DELETE** – This is used to remove a resource.
- **POST** – This is used to update a existing resource or create a new resource.

RESTful Web Services

A web service is a collection of open protocols and standards used for exchanging data between applications or systems. Software applications written in various programming languages and running on various platforms can use web services to exchange data over computer networks like the Internet in a manner similar to inter-process communication on a single computer. This interoperability (e.g., communication between Java and Python, or Windows and Linux applications) is due to the use of open standards.

Web services based on REST Architecture are known as RESTful web services. These webservices uses HTTP methods to implement the concept of REST architecture. A RESTful web service usually defines a URI, Uniform Resource Identifier a service, which provides resource representation such as JSON and set of HTTP Methods.

Creating RESTful for A Library

Consider we have a JSON based database of users having the following users in afile **users.json**:

```
{
  "user1": {
    "name" : "mahesh", "password" : "password1", "profession" : "teacher", "id": 1
  },
  "user2": {
    "name" : "suresh", "password" : "password2", "profession" : "librarian", "id": 2
  },
  "user3": {
    "name" : "ramesh", "password" : "password3", "profession" : "clerk", "id": 3
  }
}
```

Based on this information we are going to provide following RESTful APIs.

Sr.No.	URI	HTTP Method	POST body	Result
1	listUsers	GET	empty	Show list of all the users.
2	addUser	POST	JSON String	Add details of new user.
3	deleteUser	DELETE	JSON String	Delete an existing user.
4	:id	GET	empty	Show details of a user.

List Users

Let's implement our first RESTful API **listUsers** using the following code in a server.js file –
server.js

```
var express = require('express');var app = express();

var fs = require("fs");
app.get('/listUsers', function (req, res) {

  fs.readFile(__dirname + "/" + "users.json", 'utf8', function (err, data) {console.log( data );
res.end( data );

});

})
var server = app.listen(8081, function () {var host = server.address().address
var port = server.address().port

console.log("Example app listening at http://%s:%s", host, port)

})
```

Now try to access defined API using URL: *http://127.0.0.1:8081/listUsers* and HTTP Method : *GET* on local machine using any REST client. This should produce following result –
You can change given IP address when you will put the solution in production environment.